Question 1.2.1

Numerical Integration: Euler Method Verification

Aarush Dilawri
Roll Number: EE25BTECH11001

Problem Statement

Question:

The Euler iteration formula for numerically integrating a first order nonlinear differential equation of form $\dot{x} = f(x)$, with a constant step size of Δt is (AE 2007)

Options:

a) $x_{k+1} = x_k - \Delta t \times f(x_k)$
b) $x_{k+1} = x_k + (\Delta t^2/2) \times f(x_k)$
c) $x_{k+1} = x_k - (1/\Delta t) \times f(x_k)$
d) $x_{k+1} = x_k + \Delta t \times f(x_k)$

Mathematical Foundation

Using a First-Order Taylor Series expansion around time t_k:

$$x(t_k + \Delta t) \approx x(t_k) + \Delta t \cdot \left.\frac{dx}{dt}\right|_{t_k}$$

Given our differential equation is $\frac{dx}{dt} = f(x)$, we substitute the derivative:

$$x(t_{k+1}) \approx x(t_k) + \Delta t \cdot f(x(t_k))$$

Converting this to discrete iterative notation yields the Forward Euler formula:

Euler Iteration Formula

$$x_{k+1} = x_k + \Delta t \cdot f(x_k)$$

Verification: Testing the Formula

To verify this iteration formula, we will apply it to three different differential equations (including nonlinear ones) and compare the numerical iteration against the exact analytical solution.

Test Case 1: Linear Decay

- ODE: $\dot{x} = -x \implies f(x) = -x$
- True Solution: $x(t) = e^{-t}$

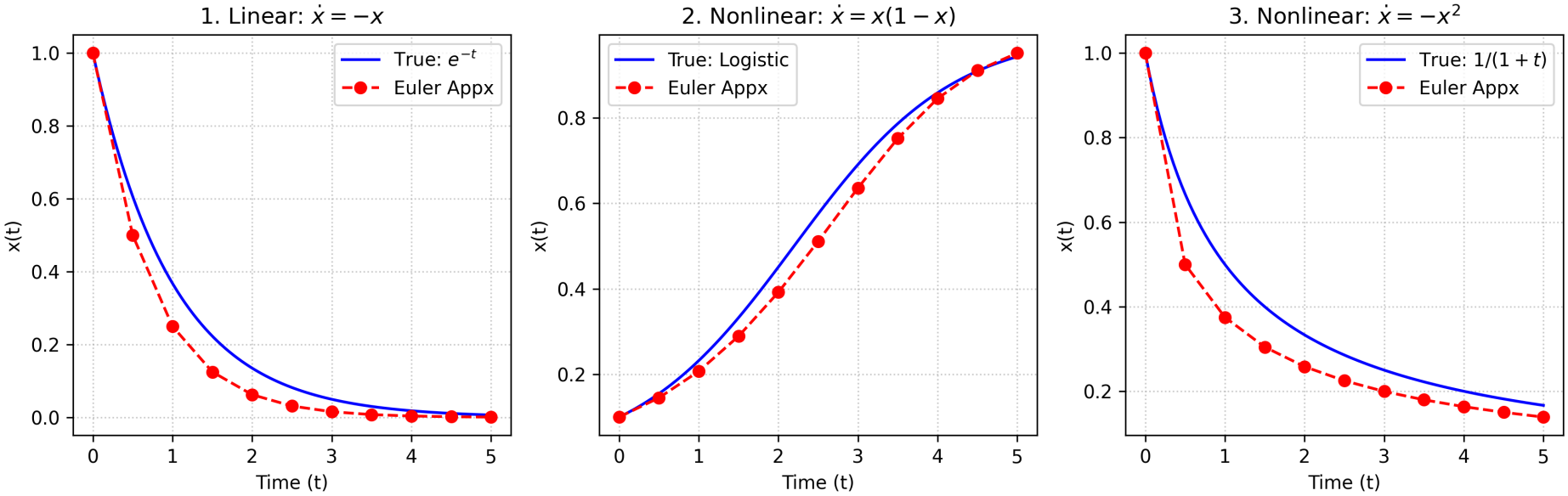
Test Case 2: Logistic Growth (Nonlinear)

- ODE: $\dot{x} = x(1-x) \implies f(x) = x(1-x)$
- True Solution: $x(t) = \frac{1}{1+9e^{-t}} \quad (\text{for } x_0 = 0.1)$

Test Case 3: Quadratic Decay (Nonlinear)

- ODE: $\dot{x} = -x^2 \implies f(x) = -x^2$
- True Solution: $x(t) = \frac{1}{1+t} \quad (\text{for } x_0 = 1)$

Graphical Verification of Euler Iteration



Python Verification Script

```python
import numpy as np
import matplotlib.pyplot as plt

dt, t_end = 0.5, 5.0
t_true = np.linspace(0, t_end, 100)
t_euler = np.arange(0, t_end + dt, dt)
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

# 1. Linear: x' = -x
x_e1 = [1.0]
for _ in t_euler[:-1]: x_e1.append(x_e1[-1] + dt * (-x_e1[-1]))
axes[0].plot(t_true, np.exp(-t_true), 'b-', label='True')
axes[0].plot(t_euler, x_e1, 'ro--', label='Euler Appx')
axes[0].set_title(r'1. Linear: $\dot{x} = -x$')

# 2. Logistic Growth: x' = x(1-x)
x_e2 = [0.1]
for _ in t_euler[:-1]: x_e2.append(x_e2[-1] + dt * (x_e2[-1]*(1-x_e2[-1])))
axes[1].plot(t_true, 1/(1+9*np.exp(-t_true)), 'b-', label='True')
axes[1].plot(t_euler, x_e2, 'ro--', label='Euler Appx')
axes[1].set_title(r'2. Nonlinear: $\dot{x} = x(1-x)$')

# 3. Quadratic Decay: x' = -x^2
x_e3 = [1.0]
for _ in t_euler[:-1]: x_e3.append(x_e3[-1] + dt * (-x_e3[-1]**2))
axes[2].plot(t_true, 1/(1+t_true), 'b-', label='True')
axes[2].plot(t_euler, x_e3, 'ro--', label='Euler Appx')
axes[2].set_title(r'3. Nonlinear: $\dot{x} = -x^2$')
```

Python Verification Script

```
for ax in axes: ax.legend(); ax.grid(True, linestyle=':', alpha=0.7)
plt.tight_layout()
plt.savefig('euler_verification.png')
```

Conclusion

As verified by the numerical plots across linear and nonlinear test cases, the formula successfully approximates the true analytical solutions by taking discrete tangent steps.

Final Answer

The correct option is **(d)** $x_{k+1} = x_k + \Delta t \times f(x_k)$.